R1Pro Airport Tray Manipulation

Project Review and Operations Handover

English edition | 16 October 2026 | Internal engineering handover

This report first reviews the project's exploratory work, observed results and unresolved questions. It then documents the environment, data generation, training, inference services and robot-side operating procedures required for handover. The final appendices enumerate the applicable scripts and configuration files.

The project has established reproducible simulation assets, contact-aware expert collection, native camera recording, data provenance checks and several trained policy variants. A complete native Isaac Sim three-tray expert demonstration and a separate single-pose, three-second Pick/Place benchmark have been independently audited. Learned-policy reliability remains unresolved: the matched real-only versus mixed-policy closed-loop study did not complete either action successfully under its registered protocol.

Evidence boundary. This edition uses repository artifacts available at 13:16 SGT on 16 October 2026. The operational environment snapshot was captured at 12:44 SGT; the latest collection status cited here was captured at 13:13:54 SGT. This document is a dated handover, not a live status service. No training, simulation or robot motion was initiated to produce it.

Reading convention. A completed process, saved checkpoint, offline metric, expert demonstration and policy task success are distinct claims. References [R01]-[R18] identify the supporting files. All relative paths are relative to the project root. Command blocks marked as new-run recipes require reviewed inputs and a fresh output directory.

1. Exploration summary

The work progressed from MuJoCo task construction and expert control to policy comparisons, real-data transfer, real-command replay and native PhysX validation. These tracks share tooling but have different task contracts, datasets and success criteria. Their success rates must not be pooled.

Workstream	Work performed and observed outcome	Interpretation
Historical MuJoCo control	Scene preparation, front-of-table reachability, bimanual contact transfer, timing and failure analysis; historical full runs reported 180/400 and 193/400 successful sources.	Historical task versions; no evidence of the proposed 70% full-task target. [R02]
Three-table, single-tray task	Three exact actions; qualified V4 data with 100/10/10 complete train/val/test sources and 300/30/30 action segments.	A reachable, preselected collection; not a success estimate over every scene. [R03]
Pickup120 policy comparison	pi0.5: 50/50 operational pickups; project AffordVLA adaptation: 0/50. Both: 0/50 under the separate 18 cm / 5 deg terminal check.	Pickup-only comparison on same-parent held-out poses; no full-task or architecture-wide conclusion. [R04]
Real-data and mixed BC	Real-only, 90/10 and 50/50 variants completed 10,000 updates. Matched offline prediction metrics and independent closed-loop tests were recorded.	Offline gains did not establish reliable contact-rich task execution. [R05]-[R08]
Real-command replay	Source trajectory, torso, joint limits, gripper semantics, collision geometry and estimated registration investigated.	Pick motion was reproduced in selected runs; stable initial holding and complete Place reproduction remain unresolved. [R09]
Native Isaac Sim	Imported assets and drives audited; complete three-tray expert demo passed with four native camera streams and recorded PhysX substeps.	Expert collection success; native learned-policy task success remains pending. [R10]
Fast contact stability	Interpolation, closure timing, pad orientation and preload investigated; recorded v9 center-pose Pick and Place passed at 3 seconds each.	One expert-controlled pose; preparation excluded; no multi-pose robustness claim. [R11]
Robot deployment	Model adapters, operator command stop, LAN transport, recording compatibility and SDK diagnostics implemented.	Operational infrastructure and historical trials; no verified complete real-robot task success. [R13]

The most useful result of this exploration is a chain of inspectable evidence: source identity, physical trajectory, contact measurements, native media, split assignment, training inputs and real process exits. The next development decisions should use that chain rather than aggregate labels such as "complete".

Contents

1. Exploration summary . . . 2
2. Historical control and task development . . . 4
3. Policy learning and simulation diagnostics . . . 5
4. Real-data training and offline comparison . . . 6
5. Closed-loop transfer and the 50/50 branch . . . 7
6. Real-command replay and native simulation . . . 8
7. Contact stability and the latest branch . . . 9
8. Project layout, access and environments . . . 10
9. Installation and reproducibility . . . 11
10. Preflight, job ownership and recovery . . . 12
11. MuJoCo scene generation and collection . . . 13
12. Native Isaac assets and complete episodes . . . 14
13. Dataset freezing and specialized collection . . . 15
14. Training entry points and new runs . . . 16
15. Native BC restoration and policy evaluation . . . 17
16. Start the workstation inference services . . . 18
17. Operator control and robot LAN transport . . . 19
18. Robot-side startup and recorded execution . . . 20
19. Robot execution contract and troubleshooting . . . 21
20. Replay, monitoring and maintenance . . . 22
21. Acceptance, ownership and continuing the project . . . 23
22. Reference register . . . 24
Appendix A. Script inventory . . . 25
Appendix B. Configuration inventory . . . 39

2. Historical control and task development

2.1 MuJoCo baseline and lessons

Early work established the robot and airport scene assets, multi-camera rendering, mobile-base operation, inverse-kinematics planning for expert control and a state machine for tray manipulation. Experiments varied base station and yaw, bimanual grasp feasibility, clearance, supported release, hand-to-hand transfer and action timing. The controller used physical contacts and preserved failures for diagnosis. [R01]-[R03]

Historical v36 and v64 campaigns reported 180/400 (45.00%) and 193/400 (48.25%) completed sources. These are results under their own task versions and acceptance rules. Changes to acceptance tolerances must be reported separately from control improvements. Neither result demonstrates 70% success under the current contract. The earlier v50 evidence covered only three scenes, with two successes, and is not the same scene version as the current local resources. [R02]

The legacy successful-source export contains 149 train, 22 validation and 22 test sources. Its eight-text segmentation produced 2,654 / 390 / 381 segments. These counts must not be reused as current three-action or native Isaac dataset counts. [R03]

2.2 Current three-action contract

The main MuJoCo task preserves the three-table layout and dimensions and uses one tray. Base position, base yaw and both grasp poses are searched jointly at the front of the source and destination tables. Movement between tables is allowed; side-of-table manipulation is excluded. The only task strings are:

- `Pick up the tray`
- `Place the tray`
- `Flush tray to the left`

For empty-tray Pick and Place, the arms grasp opposing long edges. The tray long axis is approximately parallel to the robot's horizontal forward direction while facing the table. Any necessary single-arm pre-adjustment remains part of Pick. After Place, the environment loads a backpack, coat and laptop; loading is not a training action.

Loaded Flush follows right push, stable left grasp, right release, physical leftward tray movement of at least 3 cm, then left release with stable support. Reaching the extreme left edge is no longer required. Existing authorized position, boundary and base-translation tolerances, release proximity and timing exclusions are defined by the task specification; this handover introduces no new thresholds. [R01]

Only the specifically authorized visual obstruction at the wrist cameras is removed. Gripper appearance, fingers, collisions, inertia and camera orientation remain preserved. Native four-camera evidence and source-based split assignment remain part of qualification.

3. Policy learning and simulation diagnostics

3.1 Single-tray V4 data and initial BC

V4 contains 120 qualified complete sources: 100 train, 10 validation and 10 test, corresponding to 300 / 30 / 30 three-action segments. The sources were selected for reachable initial conditions; collection completion does not imply universal scene reachability. The separate `three_actions_bc_20261008_v1` run saved a checkpoint after 10 new updates. A subsequent evaluation stage failed with `actual_zero_frame_initialization_evidence_required`. Later milestones were not established by that run. [R02, R03]

3.2 Pickup120 comparison

Both models used 120 training pickup episodes and a fixed 50-case, same-parent pose holdout. The pi0.5 model completed 10,000 action BC updates. The project AffordVLA adaptation completed 1,000 mask warmup updates followed by 10,000 action BC updates. Neither evaluation used a teacher, inverse-kinematics solver or inverse-dynamics fallback. [R04]

Measure	pi0.5	Project AffordVLA adaptation
Operational pickup result	50 / 50	0 / 50
Separate 18 cm / 5 deg terminal check	0 / 50	0 / 50
Recorded inference calls	1,738	5,000
Main observed behavior	Physical two-hand contact and table separation	No effective tray lift

The AffordVLA branch uses Qwen3-VL-4B, visual-region pooling, a flow action head and text LoRA. Its mask supervision is the simulation-visible target-tray silhouette. It is not a complete reproduction of the original research method, does not use RAGNet and does not specifically label long-edge grasp regions. The failure mechanism has not been isolated; architecture, annotation, normalization and action-interface explanations remain hypotheses until tested.

An older evaluation implementation error and an earlier CUDA compatibility failure are retained as separate failed attempts. They must not be relabeled as model capability results. The final 50-case result supersedes intermediate progress snapshots such as 11/50.

3.3 Free-run diagnostic

The independent free-run diagnostic recorded 600 seconds, 12,000 frames, 2,400 inference calls and 300,000 physical substeps. The tray fell, opposed long-edge grasping was absent and base XY displacement reached approximately 0.952 m. The recording completed, but the task failed. This run is diagnostic evidence and is excluded from formal training and aggregate task-success reporting. [R02]

4. Real-data training and offline comparison

4.1 Provenance and representation

The supplied real dataset contains 134 independent action episodes: 79 Pick and 55 Place. Grouping by original recording hash yields 102 train, 13 validation and 19 test episodes. The action breakdown is 60/42, 8/5 and 11/8 respectively. At the common 20 Hz cache timeline, the splits contain 18,853 / 2,461 / 3,653 frames. Source data at 30 Hz are resampled using a shared time index; raw recordings remain preserved. [R03, R05]

The learned `mobile19` representation contains seven left-arm joints, left gripper, seven right-arm joints, right gripper and three base channels. Arm angles are radians and model gripper opening is meters. Hardware gripper opening is millimeters. Base channels are excluded from the reported real-data offline MSE; the policy representation does not predict the torso. Three 224 x 224 letterboxed views are model inputs; retaining four original camera streams is a separate data obligation.

4.2 Training variants

Variant	Initialization and training scope	Verified state
Real-only	Official `pi05_base`; real training data only	10,000 updates; final checkpoint index 9999; offline evaluation complete
90/10 mixed	Verified simulation-trained parent; new optimizer; real-heavy sampling	10,000 updates; saved final checkpoint; offline and registered simulation tests
50/50 mixed	Same verified simulation parent as the 90/10 branch; new optimizer; batch 8 = 4 real + 4 simulation	10,000 updates; prepare/train/evaluate exits 0; separate run and model

Real-only and mixed models have different initialization histories. A comparison between them is not a strict one-factor mixing-ratio ablation. Sampling percentages refer to training draws, not episode counts or recording duration. Normalization statistics are computed from training inputs only. [R05, R08]

4.3 Matched full-frame offline study

The paired study used the same 19 test episodes, all 3,653 frames, horizon 20 and matched per-frame noise for each model. It contains 69,450 valid horizon targets per model. All 104 PNG and 104 PDF plots were generated and independently checked. [R06]

Model	Pick arm RMSE, deg	Place arm RMSE, deg	Pick gripper RMSE, mm	Place gripper RMSE, mm
Real-only	5.349	2.431	15.213	29.209
90/10 mixed	5.235	2.405	14.118	28.666

The mixed model had lower aggregate errors on these reported measures. This supports a limited offline prediction comparison. It does not establish stable contact, task completion or real-robot reliability. Earlier stride-5 evaluation numbers have a different sampling protocol and must not be merged into this table.

5. Closed-loop transfer and the 50/50 branch

5.1 Matched independent-action simulation

The registered comparison used each saved 10,000-update real-only and 90/10 mixed model for ten Pick and ten Place cases. Models received paired recorded physical initial states and matched noise. Each independent action had a 50-second simulation budget. There were no teacher calls, IK calls, inverse-solver calls or online state resets. [R07]

Model	Pick	Place	Completed test executions
Real-only	0 / 10	0 / 10	20
90/10 mixed	0 / 10	0 / 10	20

All 40 executions closed with 160 camera videos at 224 x 224 and 20 Hz and 40 traces. The delivery audit checked frame counts, hashes, paired initial conditions, finite model outputs and physical substeps. Typical failure labels were `no_stable_opposed_lift` and `no_stable_target_support_release_alignment`.

These were independent actions initialized from recorded states, not a continuous full-task rollout. Their result demonstrates a gap between offline action prediction and reliable closed-loop contact manipulation under this protocol. It does not identify a unique causal mechanism. An older mixed-model 2/10 pickup result used a different protocol and remains a separate historical result.

5.2 Completed 50/50 experiment

The 50/50 run reuses 102 real training episodes and ten qualified V4 simulation training sources, the latter providing 30 action segments and 13,401 frames. Across 10,000 updates, the sample schedule draws 40,000 real and 40,000 simulation samples. Sampling within each domain is action-balanced. Real data contain Pick/Place; simulation includes Pick/Place/Flush. [R08]

Offline test subset	Arm MSE, rad squared	Gripper MSE, mm squared
Pick, 11 episodes	0.0086941904	199.7232532
Place, 8 episodes	0.0018802719	755.9309904

These are the run's test-19, horizon-20 offline metrics. They should be interpreted with that run's evaluation sampling and normalization records. No closed-loop simulation or real-robot task-success result has been established for this model in the evidence used here. The robot service named `mixed` still refers to the earlier 90/10 model; the 50/50 checkpoint was not automatically deployed.

5.3 Decisions still requiring experimental evidence

Further work should distinguish perception and action prediction errors from initialization, timing, contact, calibration and execution differences. Offline MSE alone cannot determine which mechanism prevents stable grasping. Changes to training design, grasp strategy, base range or acceptance criteria require an explicit documented decision before a new experiment is treated as comparable.

6. Real-command replay and native simulation

6.1 Real2sim investigation

The replay track preserves the original 20-channel trajectory, including torso motion, and uses a source-specific URDF with explicit joint limits and gripper semantics. It investigates source timing, initial object registration, finger origin/axis mapping, collision decomposition and drive response. The four-finger collision package contains 332 convex components. Registration used in these experiments is estimated and source-bound; it is not a verified measurement of the physical workspace. [R09]

The execution-alignment and held-contact revisions reproduced a selected Pick tray-height increase of approximately 19.55 cm. Place still lost its initial grasp during the interval that should have remained supported by the hands. Reducing initial penetration and eliminating terminal snagging did not establish full Place reproduction. A final state with table support and released fingers cannot erase the earlier loss of the object.

Contacting the table before opening the grippers is not, by itself, a failure: normal placement can do that. The relevant failure evidence is loss of the intended initial grasp and an uncontrolled drop. Earlier probe classification errors were corrected and retained. Differences in drive response were observed, but actuator dynamics and sensor latency were not identified sufficiently to establish a single cause.

Replay completion and exit code 0 therefore indicate completed execution, not successful task reproduction. These diagnostic runs are not training data. Isaac and Genesis replay adapters initially received interface tests only; later native Isaac task results below belong to a separate branch. No native Genesis task-success result is claimed.

6.2 Native Isaac Sim three-tray demonstration

Native development included USD import, source mass/inertia checks, base drives, the second gripper finger's mimic behavior, table contact filtering, payload behavior and camera transforms. A source tray mass of 1.2 kg was restored after an incorrect import produced 27.733 kg. Import success and native stepping were treated as prerequisites, not task completion. [R10, R14]

The independently audited `demo_candidate_12` completed all three tray cycles. It produced one complete source and nine action segments, four native videos with 5,462 frames each at 20 Hz, and 136,550 recorded PhysX substeps. Actual leftward movements after handover were 3.248 / 3.120 / 3.359 cm. All three payload sets remained on their trays, final grippers were released and final boundaries passed. [R10]

The demonstration used an expert collection controller. Private teacher kinematics planned commands; physical motion and contact came from PhysX. It is evidence of native expert task execution, not learned-policy performance. The first registered training batch contains one complete source per train/validation/test split; nine segments per source imply 27 action segments and 108 four-camera clip files, subject to the frozen manifest and media audit.

7. Contact stability and the latest branch

7.1 What the fast Pick/Place attempts established

Earlier fast runs met endpoint checks while exhibiting substantial rotation during motion. Subsequent experiments investigated interpolation and coordinated rate limits, gradual closure, torsional contact, contact-pad centers, grasp offsets and pad-normal orientation. A pad-orientation revision alone still produced zero left-hand contact. The successful subsequent candidate combined aligned pad normals, an 8 mm closure target and two physical response frames. These are simulation control settings, not measured rubber calibration. [R11]

Recorded v9 center-pose measure	Pick	Place
Action duration	3.000 s	3.000 s
Maximum tray inclination	1.259 deg	1.407 deg
Maximum angular velocity	10.50 deg/s	17.05 deg/s
Relative gripper/tray displacement	4.75 mm	5.33 mm
Contact/end-state evidence	17.48 cm lift; 5.65 / 5.87 N finger forces	4.674 mm release gap; 11.67 N table support; both fingers released

The full native recording has five streams of 259 frames at 20 Hz and 6,475 physical substeps. The 6.95 seconds of navigation and arm preparation before Pick are excluded from the two three-second action budgets. Relative displacement includes both slip and compliance; it is not a calibrated material-slip measurement. The maximum target joint rate, including the first transition, was 1.5 rad/s.

This result is one independently passed center-pose expert case. It does not establish multi-pose robustness, a complete three-tray task or model success, and it is not eligible for training. Historical videos and failed candidates retain their original labels. Earlier human visual review covered keyframes, while full-motion stability was checked quantitatively from the trace; the record does not claim continuous human inspection of every video frame. [R11]

7.2 Latest recorded status at the evidence cutoff

The original native BC was restored in a separate `bc_resume_after_stable_20261016_v1` run after the recorded-video review. It retains the original frozen data and parent model, verifies normalization and sample-index hashes, and corrects the training subprocess path to the existing Conda CUDA compiler. The latest cited status remains initialization with zero actual updates. Native model evaluation remains pending. [R12]

A new fixed-base Pick/Place branch uses `runs/isaacsim_fixed_pickplace_20261016_v1/code35`. Each pair physically continues from Pick into Place without resetting to a canonical held pose. Formal clips use fixed base commands while varying arm state, tray XY/yaw and lift height. Initial candidates are 14 cm and 18 cm; a separate 20 cm / +5 deg candidate failed path preflight and remains recorded. This is expert collection, not VLA-generated data or DAgger.

At 13:13:54 SGT, the first candidate had 175 recorded progress frames and zero closed pairs. The two planned pairs inherit the train split of parent `airport_0004_20270920`; validation/test splits are not fabricated from them. No new two-action training is implied. This branch is independent of the three-action native BC. [R12]

8. Project layout, access and environments

8.1 Direct server access

From a Windows terminal, connect directly using either hostname or IP:

```
ssh michelle@xinmatrix
# If hostname resolution is unavailable:
ssh michelle@100.104.0.108
```

The username is `michelle`, the host is `xinmatrix`, the address is `100.104.0.108` and the SSH port is 22. Enter the separately supplied password interactively when prompted. Verify host identity on first connection. Credentials are not embedded in this document or its command examples. The client must be able to reach the address. Distinguish name-resolution, network and authentication failures; loss of connectivity does not establish that remote work stopped.

8.2 Machines and working directories

Machine	Directory and responsibility
Windows workspace	`C:/Users/HAOZEW/Documents/code/r1pro-airport-sim`; source, documentation and small JSON/TXT evidence
GPU workstation	`/home/michelle/haoze/r1pro-airport-sim`; physics, recordings, training, checkpoints and model services
Robot terminal	`/home/nvidia/GalaxeaVLA/experiments/r1pro`; deployed client, ROS/SDK environment and local recording

The robot's current login address and hardware state were not reverified for this edition. Robot commands below assume an already logged-in, authorized on-site terminal.

8.3 Common server shell variables

```
export B=/home/michelle/haoze/r1pro-airport-sim
export CODE="$B"
export SIM_PY=/home/michelle/miniconda3/envs/r1pro-tray/bin/python
export VLA_ENV=/home/michelle/miniconda3/envs/r1pro-vla
export VLA_PY="$VLA_ENV/bin/python"
export ISAAC_PY=/home/michelle/miniconda3/envs/env_isaaclab6/bin/python
export AIRPORT_PROJECT="$B"
export PYTHONPATH="$CODE${PYTHONPATH:+:$PYTHONPATH}"
cd "$B"
```

`B` is the resource and data root. `CODE` is the reviewed code root for the chosen run. For a frozen experiment, use the code identity in its launch receipt rather than assuming the current checkout is equivalent.

Top-level responsibilities: `r1pro_teleop/` contains implementation; `scripts/` exposes workflows and diagnostics; `configs/` stores recipes; `resources/` and `outputs/r1pro_tray_scene/` contain project assets; `runs/` holds versioned evidence; deployed `releases/` are frozen code copies. `docs/SCENE_TASK_SPEC.md` is the task contract, while `docs/project/` provides module, data and experiment navigation. [R01, R14, R15]

9. Installation and reproducibility

9.1 Verified environment snapshot

The following versions were read from the workstation at 12:44 SGT on 16 October 2026. They describe separate environments, not one combined installation recipe. [R14]

Environment	Python	Key installed packages	Use
`r1pro-tray`	3.11.16	MuJoCo 3.13.0; torch 2.8.0+cu129; LeRobot 0.4.4	MuJoCo, preparation and media validation
`r1pro-vla`	3.11.16	JAX/jaxlib 0.5.3; torch 2.7.1; LeRobot 0.1.0	Existing OpenPI training and inference
`env_isaaclab6`	3.12.13	Isaac Sim 6.0.1.0; torch 2.11.0+cu130; MuJoCo 3.8.0	Native PhysX; MuJoCo only for private teacher kinematics/asset reading

9.2 New MuJoCo environment

```
conda create -n r1pro-tray python=3.11 -y
conda activate r1pro-tray
cd /path/to/r1pro-airport-sim
bash scripts/setup_r1pro_server.sh --with-lerobot
python -m pip install -r requirements-experiments.txt
python -m pip check
python scripts/project.py check --compile
```

Install only inside a non-base Conda environment. The simulation requirements do not recreate OpenPI or Isaac. `pip install -e .` alone does not install the complete runtime. Windows supports asset inspection and some tests; production background collection relies on Linux facilities such as `/proc`, `fcntl` and EGL.

OpenPI needs project-local `third_party/openpi`, its `src` and `packages/openpi-client/src`, a matching JAX environment and verified parent weights. Isaac needs the existing native environment, imported USD assets and drive audits. There is no validated single-command rebuild of the entire workstation.

9.3 Migration package

Before migration, save each environment's `conda list --explicit` and `python -m pip freeze`, the OpenPI revision where Git metadata exists, and resource/model/data hashes. Recreate environments at the target prefix instead of copying Conda directories or modifying vendor code.

Preserve source, configs, scene assets, resources, OpenPI source, legal datasets, parent split registries, model weights, normalization, frozen releases, launch identities and actual exit receipts. Existing collections contain absolute paths; a relocated dataset requires a new collection identity rather than edited old source fingerprints. All required resources must remain within the project; execution must not depend on the original humanvideo directory.

Large video and trajectory artifacts remain on the server by default. Handover should include their exact path indexes and hashes, while transferring JSON/TXT metadata locally. The live robot environment remains governed by the deployed `start.sh` and the site's verified hardware procedure.

10. Preflight, job ownership and recovery

10.1 Read-only preflight

After setting the variables in Section 8, inspect the project and resources:

```
"$SIM_PY" scripts/project.py check
"$SIM_PY" scripts/sync_task_spec.py --check
nvidia-smi
ps -eo pid,lstart,etime,args | grep -E 'python|isaac|train|supervise'
ss -ltnp
```

For an existing job, match the PID, start ticks, command line and working directory to its launch record, then inspect the stage, heartbeat, logs and real child exit receipt. A PID, a busy GPU or a status file alone is insufficient. Healthy jobs must not be duplicated, and unrelated services must not be stopped.

10.2 Auditable stage execution

`isaacsim_stage_runner.py` accepts a JSON job with `argv` as an array, `cwd`, `log`, `launch`, `receipt` and optional `environment`. Copy `docs/handoff/examples/stage_job.example.json` into a fresh run, replace every placeholder, and create the log parent directory before launch.

```
: "${JOB:?set JOB to the reviewed new job.json}"
"$SIM_PY" "$CODE/scripts/isaacsim_stage_runner.py" --job "$JOB"
```

The runner waits for the actual child process and records its exit. It does not provide business-success validation or automatic single-instance protection. For detached operation, use the appropriate supervisor/guardian and its lock, with launch identity and independent logs. A successful `nohup` return or `$!` value is not task completion.

Use a new run for changed code, settings or resources. Preserve failed attempts. Existing bounded recovery is limited to three attempts with the supervisor's backoff policy; it is not permission to retry arbitrary physical candidates indefinitely. Resume only from a validated saved state with the original data and optimizer semantics.

10.3 CUDA compiler checks

The native BC branch encountered a subprocess path problem that selected the system CUDA compiler instead of the existing Conda compiler. Preserve the reviewed environment in both the parent shell and the job JSON:

```
export PATH="$VLA_ENV/bin:$PATH"
command -v ptxas
ptxas --version
"$VLA_PY" -c 'import jax; print(jax.devices())'
```

Verify the expected compiler from the output. Do not modify system CUDA. Preserve the run's visible-GPU and XLA allocation settings; do not silently convert a two-GPU recipe to one GPU or change batch size to work around memory exhaustion.

11. MuJoCo scene generation and collection

11.1 One new complete source

`run_three_table_single_tray.py` is an early empty-tray diagnostic. The full-data entry point is `collect_three_table_full_source.py`, or its registered campaign/workflow. The following recipe writes a new scene and runs physical simulation:

```
"$SIM_PY" scripts/generate_three_table_single_tray.py \
  --output "$B/runs/NEW_SINGLE_SCENE" --seed 42

# Use the generated scene JSON and reviewed task inputs.
: "${SCENE:?set SCENE}"
: "${TASK_CONFIG:?set TASK_CONFIG}"
: "${PUSH_REFERENCE:?set PUSH_REFERENCE}"
"$SIM_PY" scripts/collect_three_table_full_source.py \
  --scene "$SCENE" --config "$TASK_CONFIG" \
  --push-reference-scene "$PUSH_REFERENCE" \
  --output "$B/runs/NEW_SINGLE_SOURCE"
```

Default parent sources are registered under `resources/single_tray_three_table_v1/source_registry`. If replacing that lineage, explicitly provide `--parent-scene`, `--parent-manifest` and `--split-registry`. A random seed does not authorize split reassignment; the same parent scene cannot cross train/test boundaries. [R01, R03, R15]

11.2 Batch workflows

Entry point	Role and required distinction
`plan_three_table_reachable_initial_poses.py`	Registers reachable candidates; planning is not physical success
`collect_three_table_campaign.py`	Requires selection, inventory, parent manifest, split registry and output; default is a plan; `--execute` with config and push-reference-scene runs physics; `--recover` is explicit
`supervise_three_table_workflow.py`	Runs a complete workflow from `--options`; validate the exact options schema and launch identity
`supervise_three_table_single_tray_training.py`	Data/training workflow from `--options`; `--data-only` suppresses training
`supervise_verified_three_action_training.py`	Qualified V4 three-action BC path; do not substitute historical eight-text data

Options files are workflow-specific, not interchangeable. When copying a frozen recipe, inspect every input/output, source domain, camera, split, interpreter and parent checkpoint; changing only the output path is insufficient.

11.3 Qualified action export

```
: "${QUALIFIED_RECORDS:?set validated full-source JSONL}"
"$SIM_PY" scripts/export_three_table_single_tray_actions.py \
  --records "$QUALIFIED_RECORDS" \
  --destination "$B/runs/NEW_ACTION3_TRAIN" \
  --repo-id airport/new_action3_train --split train
```

Export validation and test separately. The records file is qualified complete-source JSONL, not an arbitrary list of episode paths. Exclude environment loading from action clips. Verify source count, segment count, four-camera file count, FPS, dimensions, frame alignment, hashes, train-only normalization and the intended dataset loader before training.

12. Native Isaac assets and complete episodes

12.1 Prepare and audit assets

The legacy 193 successful MuJoCo sources are inputs to discovery and conversion, not native Isaac training data. Use the reviewed payload template and a new asset run:

```
: "${PAYLOAD_TEMPLATE:?set reviewed payload-template JSON}"
"$SIM_PY" scripts/supervise_isaacsim_three_tray_assets.py \
  --run "$B/runs/NEW_ISAAC_ASSETS" --project "$B" \
  --legacy-root "$B/runs/vla_20261005/task_complete_actions_v1" \
  --payload-template "$PAYLOAD_TEMPLATE" \
  --isaac-python "$ISAAC_PY" --source-python "$SIM_PY"
```

The component checks include `isaacsim_scene_smoke.py --xml ... --output ...` and `isaacsim_control_preflight.py --usd ... --xml ... --output ...`. Inspect mass/inertia, joint mapping, finger mimic, base drives, table contacts, camera transforms and payloads. Optional asset flags must follow the registered conversion plan. Successful import is not successful control.

12.2 Execute and qualify a new native episode

```
: "${ISAAC_SCENE_ROOT:?set verified native scene root}"
"$ISAAC_PY" "$CODE/scripts/run_isaacsim_three_tray_task.py" \
  --scene-root "$ISAAC_SCENE_ROOT" \
  --output "$B/runs/NEW_ISAAC_EPISODE" \
  --seed 42 --episode-id new_unique_episode_id
```

For formal collection, wrap the command in the stage runner and write the actual child receipt to the episode's `process_exit.json`. Create the parent directory first; let the task create its new episode directory. `--no-video-diagnostic` intentionally omits media and must remain ineligible for training.

Expected artifacts include status/progress, `native_trace.npz`, native-step/contact logs, observer events, `episode.json`, `qualification.json`, four MP4 files, encoder exits and any failure evidence. Native trace timing distinguishes pre-action RGB/state from the completed physical interval. Preserve this convention when checking frame alignment.

```
: "${NATIVE_REGISTRY:?set parent split registry}"
"$SIM_PY" scripts/audit_isaacsim_three_tray_demo.py \
  --case "$B/runs/NEW_ISAAC_EPISODE" \
  --registry "$NATIVE_REGISTRY" \
  --output "$B/runs/NEW_ISAAC_AUDIT.json"
```

`supervise_isaacsim_three_tray_task.py --config ...` manages the registered workflow. Its guardian must adopt the real supervisor launch identity. Preserve the validated configuration's project, demo, receipt, registry, interpreters, parent checkpoint/status and case list. Without a valid learned-policy evaluation command, the workflow must report model evaluation as pending. [R10, R15]

13. Dataset freezing and specialized collection

13.1 Freeze, clip and cache

After complete native episodes pass independent qualification and cover legal train/validation/test parents, use an absolute-path JSON array of their episode directories:

```
: "${EPISODES_JSON:?set qualified episode-list JSON}"
: "${NATIVE_REGISTRY:?set registry}"
"$SIM_PY" scripts/prepare_isaacsim_dataset.py --mode freeze \
  --registry "$NATIVE_REGISTRY" --episodes "$EPISODES_JSON" \
  --output "$B/runs/NEW_ISAAC_DATA"
"$SIM_PY" scripts/prepare_isaacsim_dataset.py --mode clips \
  --registry "$NATIVE_REGISTRY" \
  --manifest "$B/runs/NEW_ISAAC_DATA/manifest.json" \
  --output "$B/runs/NEW_ISAAC_CLIPS"
"$SIM_PY" scripts/prepare_isaacsim_dataset.py --mode cache \
  --registry "$NATIVE_REGISTRY" \
  --manifest "$B/runs/NEW_ISAAC_DATA/manifest.json" \
  --output "$B/runs/NEW_ISAAC_CACHE"
```

Each complete three-tray source produces nine action segments. Count complete sources, clips and camera files separately. A single-source preview is not a three-split training batch. Later expansion uses a new frozen version and must not modify files or manifests being read by training.

13.2 Fast Pick/Place diagnostics

`run_isaacsim_fast_pickplace.py --config <case.json>` runs one case; `supervise_isaacsim_fast_pickplace.py --config <campaign.json>` dispatches a registered campaign. The case and campaign schemas differ. `audit_isaacsim_fast_pickplace.py --case ... --output ...` independently checks the result, and `finalize_isaac_fast_pickplace.py` assembles closed recordings.

Review output, scene root, seed, case ID, tray pose, lab assets and every registered control setting. With `record_cameras=false`, the output is a physics probe, not a completed video. Each three-second action has 60 native frames at 20 Hz; separately report navigation/preparation. Audit the complete trajectory, inclination, angular velocity, relative motion and contact, not only endpoint `passed` fields. [R11, R15]

13.3 Fixed-base continuous Pick/Place

`collect_isaac_fixed_pickplace.py` is the new independent branch. Use its reviewed code35 configuration and inspect its schema before launch. The output separates `dataset/pick_up_the_tray/train/case_NNN` and `dataset/place_the_tray/train/case_NNN`, while preserving the physical continuity and common parent of each pair.

Place must begin from the actual Pick endpoint, with recorded 19-channel robot state, 7D tray pose, grasp forces and frame identity. Do not reset to a canonical held pose or invent validation/test data. Freeze each pair only after independent audit and complete media export. The cited branch had no closed pair at the evidence cutoff; this manual does not turn that status into data readiness. [R12]

14. Training entry points and new runs

14.1 Select the correct training scope

Script	Interface and purpose
`train_pi05_airport.py`	`--config <TrainingSpec.json>`; `--normalize-only` computes normalization; historical mobile19 LoRA recipes
`train_pi05_accelerated.py`	`--spec ... --parent-status ... --prepare-cache`, then `--stop-after N`; `--resume` has explicit saved-state semantics
`train_pi05_isaac_three_tray.py`	`--run <training> --mode prepare/train --stop N`; native frozen inputs; two visible GPUs
`run_pi05_real_only.py`	`--run ... --mode prepare/train/evaluate/supervise`; real-only branch initialized from `pi05_base`
`run_real_sim_pickplace.py`	`--run ... --mode prepare/mse/train/simulate/supervise`; historical 90/10 workflow; includes older comparison stages
`train_pi05_real_sim50.py`	`--run ... --mode prepare/train/evaluate/supervise`; 4+4 batch; supervisor fixes GPU 0
`compare_pickup120.py` and `resume_afford_pi05_comparison.py`	Pickup120 comparison only; inspect fixed server paths and original identities

These scripts are not interchangeable general-purpose trainers. Several resolve previously frozen caches and parent models through source constants. `--run` alone does not bind arbitrary new data. Before migration or reuse, inspect inputs, integrity records, frozen selections and fixed paths. [R05, R08, R15]

14.2 New real-only experiment template

Use only when a new experiment is intended and the existing data/checkpoint hashes and GPU allocation have been verified:

```
export PATH="$VLA_ENV/bin:$PATH"
export XLA_PYTHON_CLIENT_PREALLOCATE=false
export WANDB_MODE=disabled
: "${GPU_ID:?set allocated GPU}"
export CUDA_VISIBLE_DEVICES="$GPU_ID"
"$VLA_PY" scripts/run_pi05_real_only.py \
  --run "$B/runs/NEW_REAL_ONLY" --mode supervise
```

For the 50/50 supervisor, verify that GPU 0 is available and approved, then create a new run before launch:

```
mkdir "$B/runs/NEW_MIX50"
"$VLA_PY" scripts/train_pi05_real_sim50.py \
  --run "$B/runs/NEW_MIX50" --mode supervise
```

Do not restart the already completed historical runs. Keep source domains explicit: `real_world/user_provided` and `simulation/program_generated`. Never relabel simulation as human teleoperation or real user recordings as simulation. Record actual updates, checkpoint save receipts and evaluation separately from finite loss or supervisor completion.

15. Native BC restoration and policy evaluation

15.1 Native training inputs

The native trainer's `training/inputs.json` binds five absolute paths: `cache`, `manifest`, `registry`, `parent_checkpoint` and `parent_status`. Preparation validates the three splits, action labels and parent weights and writes normalization, sample indices, integrity and preparation status.

```
: "${TRAIN_RUN:?set reviewed new training directory}"
"$VLA_PY" scripts/train_pi05_isaac_three_tray.py \
  --run "$TRAIN_RUN" --mode prepare
# Confirm both GPUs and the reviewed compiler environment first.
"$VLA_PY" scripts/train_pi05_isaac_three_tray.py \
  --run "$TRAIN_RUN" --mode train --stop 10
```

The registered milestones are 10 / 100 / 1,000 / 10,000 / 100,000 actual updates. Later stages reuse the saved state within the same training run. Checkpoint numbering is zero-based in these recipes: index 9999 corresponds to 10,000 updates only when the status and metadata agree.

15.2 Restoration after recorded-video review

`resume_isaacsim_bc_after_video.py --config <reviewed-config>` waits for explicit native-video review, checks evidence hashes and preserves the original frozen inputs, normalization and sample indices. Required configuration includes `run`, `training`, `original_training`, `review`, `project`, `compiler_bin`, `vla_python` and `training_code`.

The restoration directory is under `runs/isaacsim_three_tray_workflow_20261015_v1/bc_resume_after_stable_20261016_v1`. The original `pipeline_v15` deferred state is historical coordinator state; read `CURRENT_NATIVE_BC_RESUME.json` and the restoration run to determine actual training status. At this edition's cutoff the cited actual-update count was still zero. Do not duplicate a healthy restoration process or fabricate `approved_for_original_BC_resume`. The fast diagnostic recording is excluded from training. [R12]

15.3 Evaluation rules

`evaluate_three_table_single_tray_pi05_batch.py` uses a registered protocol, checkpoint identity, simulation/model interpreters and output path; distinguish plan mode from `--execute`. `run_vla_fixed_scene.py` and `run_three_table_pi05_free_run.py` are diagnostics. `compare_pi05_closed_loop.py` is the registered paired-model test and must retain the actual model identity.

A policy test must execute the policy without hidden teacher, IK, inverse-dynamics or corrective fallback. BC is not DAgger unless DAgger was actually performed. An independent-action test is not a full continuous task. Report simulation and real-robot results separately. Native three-tray learned-policy success remains pending until actual execution, qualification, native media and exit receipts close. [R01, R07, R15]

16. Start the workstation inference services

16.1 Endpoint and model identity

Role	Workstation endpoint	Robot LAN endpoint
`mixed`: completed 90/10 model	`100.104.0.108:8095`	`10.42.0.191:8095`
`real_only`: completed real-only model	`100.104.0.108:8096`	`10.42.0.191:8096`

The deployed adapter is `releases/galaxea_robot_deploy_20261013_v3/scripts/serve_pi05_galaxea_robot.py`. The simulation server `serve_pi05_airport.py` implements a different protocol. The 50/50 model is not automatically selected by the `mixed` endpoint. [R13, R14]

At the 12:44 SGT inventory snapshot, the LAN relay was listening, but the two model backends and keyboard controller were not found. This is a dated observation; recheck processes, addresses and GPU availability before starting anything.

16.2 Fresh output directories

The old deployment run already contains raw action chunks. Counters restart from zero, so restarting into those directories could overwrite evidence. Create a new run and copy the reviewed configurations:

```
export DEPLOY_CODE="$B/releases/galaxea_robot_deploy_20261013_v3"
export ROBOT_RUN="$B/runs/galaxea_robot_REPLACE_NEW_ID"
export OLD_ROBOT_RUN="$B/runs/galaxea_robot_deploy_20261013_v3"
mkdir "$ROBOT_RUN"
mkdir "$ROBOT_RUN/mixed" "$ROBOT_RUN/real_only"
cp "$OLD_ROBOT_RUN/mixed/server_config.json" \
  "$ROBOT_RUN/mixed/server_config.json"
cp "$OLD_ROBOT_RUN/real_only/server_config.json" \
  "$ROBOT_RUN/real_only/server_config.json"
```

Keep `training_status` bound to the verified model, `execute_steps=5`, `gripper_range_mm=[0,80]`, and matching host/port settings. Both servers use the same absolute operator-control file under `runs/galaxea_robot_deploy_20261013_v2/operator_control.json`. If that location is changed, both servers and the control terminal must be changed together. Complete examples are in `docs/handoff/examples/`.

```
: "${GPU_ID:?set allocated GPU}"
CUDA_VISIBLE_DEVICES="$GPU_ID" "$VLA_PY" \
  "$DEPLOY_CODE/scripts/serve_pi05_galaxea_robot.py" \
  --config "$ROBOT_RUN/mixed/server_config.json"
```

Start only the needed model; use a separate terminal or fully stop it before starting the other configuration. Two simultaneous models require verified memory capacity. Readiness requires completed warmup, matching checkpoint/norm identity in `server_status.json`, the actual listener and the expected process. `ready_operator_stop_latched` means ready with command stop still engaged. Background services require the same launch/exit evidence discipline as other jobs.

17. Operator control and robot LAN transport

17.1 Persistent command-stop terminal

Open one interactive server terminal from Windows:

```
ssh -t michelle@xinmatrix
# Alternatively: ssh -t michelle@100.104.0.108
```

In that server terminal, set the absolute paths and run the controller:

```
B=/home/michelle/haoze/r1pro-airport-sim
VLA_PY=/home/michelle/miniconda3/envs/r1pro-vla/bin/python
CONTROL_CODE="$B/releases/galaxea_keyboard_control_20261013_v3"
CONTROL_FILE="$B/runs/galaxea_robot_deploy_20261013_v2/operator_control.json"
"$VLA_PY" "$CONTROL_CODE/control_galaxea_servers.py" \
  --control-file "$CONTROL_FILE"
```

The default state is STOP. Only the on-site operator should press `R` after the setup is ready. `E` or Space engages stop; `Q` and terminal exit engage stop. A heartbeat older than one second causes the servers to reject commands. After rejection, rearming does not automatically restart the client; restart it deliberately.

This mechanism stops command acceptance. It cannot retract an in-flight command or the last submitted target. The hardware emergency stop remains a separate on-site control. Use one operator-control terminal and one active robot client. [R13]

17.2 LAN transport

The workstation relay listens on `10.42.0.191:8095/8096` and forwards to the model services at `100.104.0.108` on the corresponding ports. A listening TCP port does not establish model readiness. Reuse an existing healthy relay.

Only if the previous relay has exited, the address belongs to the workstation and the ports are free, create a fresh relay run. The script writes lock/status files next to itself, so execute a copy in the new run:

```
export RELAY_RUN="$B/runs/galaxea_robot_lan_REPLACE_NEW_ID"
mkdir "$RELAY_RUN"
cp "$B/scripts/forward_galaxea_lan.py" \
  "$RELAY_RUN/forward_galaxea_lan.py"
"$SIM_PY" "$RELAY_RUN/forward_galaxea_lan.py"
```

Changing the LAN requires a new explicitly configured transport version and protocol/latency verification. Do not assume a historical address is still assigned. The robot configuration files are `config_pi05_mixed_gripper80_lan_20261013.toml` and `config_pi05_real_only_gripper80_lan_20261013.toml`. Their handover copies represent the previously deployed configuration, not a fresh comparison against today's robot filesystem.

The configured client rate is 20 Hz with five action steps per inference response. Actual control cadence must be derived from recorded timestamps; the configuration alone is not a latency or throughput measurement.

18. Robot-side startup and recorded execution

18.1 Hardware and environment prerequisite

The project does not contain a verified single command for cold boot through complete robot-driver and camera startup. The site must use its established vendor/SDK startup procedure. `source start.sh` initializes the client environment and does not prove that hardware drivers are running.

In the already logged-in robot terminal, inspect the environment and topics:

```
cd /home/nvidia/GalaxeaVLA/experiments/r1pro
source ./start.sh
export FASTRTPS_DEFAULT_PROFILES_FILE=/opt/galaxea/find_server/super_client_configuration_file.xml
ros2 topic list --no-daemon
./vla status
```

A fresh shell without the DDS configuration can fail to discover topics even when the SDK exists. Verify live feedback, valid camera frames, backend readiness, the original command filter, the intended initial state and on-site stop controls before starting a client. This document records the procedure; its preparation did not authorize or execute robot motion. [R13]

18.2 Task/model selection

The diagnostic wrappers record SDK feedback and are preferred when investigating hardware faults:

```
./vla_sdk_diagnostics pick        # 90/10 mixed, Pick up the tray
./vla_sdk_diagnostics place       # 90/10 mixed, Place the tray
./vla_sdk_diagnostics pick real   # real-only
./vla_sdk_diagnostics place real
```

Regular recording-compatible entries remain `./vla pick`, `./vla place`, `./vla pick real` and `./vla place real`. These commands run inside the deployed robot directory. Do not run the repository's undeployed one-click script from the workstation as though it were the robot client.

If an alias is absent, first verify the deployed files. The v5 wrapper can be invoked directly:

```
python run_pi05_record_compat_20261013_v5.py \
  --config config_pi05_mixed_gripper80_lan_20261013.toml --record
```

Direct wrapper invocation does not switch the instruction. With no active client, set `scheduler/instruction/instruction.txt` to the exact task string. The one-click entry is preferable because it checks single-instance ownership before writing that text. Never change instructions during an active client run.

To change task/model, stop the current client with Ctrl+C, wait for recording finalization, verify exit, check stop state and the new task, then let the operator decide whether to rearm and restart. Preserve rejected commands and SDK errors; do not automatically retry physical motion.

19. Robot execution contract and troubleshooting

19.1 Units, filters and records

The original `run.py`, `scheduler/scheduler.py` and `scheduler/joint_command_filter.py` remain the execution contract. The approved filter settings are a 0.1 rad maximum joint step, 1.5 rad/s velocity, 0.25 rad tracking threshold and 0.5 s maximum observation age. Gripper opening is limited to 0-80 mm. These are inherited settings, not new limits introduced by this handover. [R13]

The model emits gripper opening in meters; the server converts to millimeters and clamps to the allowed range. It does not rescale 100 mm into 80 mm. Preserve raw model output and submitted values so saturation remains visible. Policy base channels are not deployed as hardware base commands. SDK effort units are not calibrated here and must not be reported as measured torque.

Retain server `commands.jsonl`, `raw_chunk_*.npz` and `server_status.json`; client recording directories; `joint_filter_diagnostics.jsonl`; v5 `sdk_fault_window.json`; exception logs and actual exits. NumPy recording compatibility, transport fixes and fault windows improve diagnosability but do not establish successful manipulation.

19.2 Fault isolation sequence

Symptom	First evidence to inspect	Interpretation/action
SSH connection fails	Host resolution, direct IP reachability, authentication response	Distinguish access failure from remote process state; retain last verified time
Port responds but inference fails	Backend process, warmup log, model identity, stop/heartbeat state	TCP transport may be healthy while model service is absent or stopped
ROS topics absent in a fresh shell	`start.sh`, DDS profile and discovery output	Check environment before concluding the SDK is down
Commands rejected	Observation age, joint deltas, tracking error, operator heartbeat	Preserve filter evidence; do not weaken thresholds to obtain motion
Robot stops or SDK faults	Timestamped feedback, filter diagnostics, SDK fault window, raw/submitted action	Root cause remains unproven until these signals are correlated
Recording fails	Deployed wrapper version, dependency versions and exception	Verify the compatibility wrapper; retain the failed recording attempt

19.3 Known handover gaps

The cold hardware-start procedure, today's robot-side file hashes and current sensor/SDK readiness were not revalidated for this edition. Historical filter tracking errors do not yet have a confirmed single cause. Real2sim drive and initial-grasp consistency also remain unresolved. A successor should validate these specific gaps before treating the system as unattended or production-ready.

20. Replay, monitoring and maintenance

20.1 Offline real-command replay

The replay entry point operates on recorded trajectories and does not connect to the robot. Use a new output directory, the reviewed scene and the source-specific URDF:

```
: "${REAL_MANIFEST:?set source manifest}"
: "${SCENE_XML:?set scene XML}" "${ROBOT_URDF:?set source URDF}"
"$SIM_PY" scripts/replay_real2sim.py \
  --source-manifest "$REAL_MANIFEST" --scene-xml "$SCENE_XML" \
  --urdf "$ROBOT_URDF" --output "$B/runs/NEW_REAL2SIM_REPLAY" \
  --per-task 1 --video
```

Use `--registration` and `--finger-collision-bundle` only with the reviewed source-bound assets. Companion tools include `validate_real2sim.py`, `audit_real2sim.py`, `fit_real2sim_registration.py`, `fit_real2sim_torso.py` and `diagnose_real2sim_held_pose.py`. Inspect their exact arguments and side effects. A 640 x 360, 30 fps preview is not native-resolution training media. [R09, R15]

20.2 Server-side monitoring

After direct SSH and the Section 8 setup, run the applicable read-only monitor. Omit `--watch` for one snapshot or add `--watch 10` for periodic output:

```
"$SIM_PY" scripts/watch_pi05_mix50.py \
  --run "$B/runs/real_sim50_pi05_20261015_v1"
"$SIM_PY" scripts/watch_pi05_real_only.py \
  --run "$B/runs/real_only_pi05_20261012_v1"
"$SIM_PY" scripts/watch_real_sim_transfer.py \
  --run "$B/runs/real_sim_pickplace_20261009_v3"
"$SIM_PY" scripts/watch_vla_comparison.py \
  --run "$B/runs/afford_pi05_pickup120_20261008_v2"
```

For native work, read `CURRENT_NATIVE_PIPELINE.json` and `CURRENT_NATIVE_BC_RESUME.json` under the three-tray workflow run, then inspect the selected run's status and training status. For fast diagnostics, read `runs/ISAAC_FAST_PICKPLACE_CURRENT.json` and distinguish active probes from older videos. Fixed-pair collection uses its own `collection_status.json`, producer receipt and logs. Stopping a monitor with Ctrl+C does not stop the job.

20.3 Media and historical tools

`serve_isaacsim_media.py --root <approved-run-root> --state-dir <new-state-dir> --bind 100.104.0.108 --port <free-port>` serves read-only media with Range/HEAD support. Reuse a healthy existing service. Limit its root to the intended run; do not expose the project root or credential directories.

Legacy `project.py fourcam`, `r1pro_teleop.airport_fsm` prepare/preflight/generate/export/report, action8 trainers and pickup120/200 workflows retain their original scopes. Many historical diagnostic scripts write output or execute at import time. Do not bulk-run them, including for `--help`. Use the static catalog and read source first.

21. Acceptance, ownership and continuing the project

21.1 Evidence required for each claim

Claim	Minimum supporting evidence
Job is initializing/waiting	Matching process identity, current phase/heartbeat and logs; actual updates explicitly zero where applicable
Data are ready	Qualified complete sources, native media/trace alignment, legal splits, frozen manifest and hashes
Training occurred	Actual updates greater than zero, finite metrics and timestamped process/log evidence
Model was saved	Checkpoint parameters/metadata, normalization, saved-state semantics and save receipt
Policy task succeeded	Actual policy execution, no hidden fallback, task qualification, media/trace and process exit
Workflow finished	Every required stage has a real exit and delivery audit; task outcome reported separately

Frozen collections, releases, manifests and checkpoints are immutable evidence. Use new run directories for implementation, configuration and resource changes. Preserve failure artifacts and old success labels. After core control checks, complete the relevant physical episode; static tests or low loss do not replace server execution. [R01, R16]

21.2 Recommended continuation order

First inspect the current native BC restoration and fixed-pair collection without starting duplicates. Confirm whether initialization progressed into actual updates and whether continuous Pick/Place pairs closed with full media and correct source lineage. Then assess the registered native learned-policy evaluation when its checkpoint exists. Separately resolve the real2sim initial holding state and the hardware-start/feedback gaps before extending real-robot execution.

The ordering above is a handover recommendation, not a statement that those experiments have been run or an authorization to change their design. Any proposed change to action sequence, grasping, base range, tolerances or training strategy should state the observed failure, established evidence, hypotheses and alternatives before a decision is recorded.

21.3 Documentation and interaction records

Read the root README, documentation index, task specification, current memory, decisions and module/data/experiment pages when resuming work. Implementation changes affecting generation, state machines, contacts, cameras or export require a corresponding human-reviewed task-spec update and non-base `sync_task_spec.py --write` followed by `--check`.

Refresh the static handover catalog with `scripts/build_handoff_catalog.py --write`; check it with `--check`. It parses source without importing or executing historical scripts. The English PDF is generated from this editable English source and a frozen inventory snapshot. Its source hashes and validation receipt accompany the output.

At the end of meaningful work, update the relevant topic page and current snapshot, append decisions with replacement relationships, and run `scripts/export_project_interactions.py` using the non-base project interpreter. Preserve the current project thread and its parent thread IDs. The export is an explicit invocation, not a permanently running service; credentials and private keys must not enter the archive. [R16]

22. Reference register

References below distinguish primary artifacts from derived navigation. Paths identify the evidence used in this edition; they are not instructions to run historical experiments. The companion source manifest records SHA-256 hashes at capture. Current operational status must be rechecked on the server.

ID	Evidence / navigation path	Role
R01	`docs/SCENE_TASK_SPEC.md`	Current task contract; use the captured hash/version for this edition
R02	`docs/project/EXPERIMENT_STATUS.md` and `docs/governance/README_20261016_BEFORE.md`	Derived historical experiment summaries; old large-run results are reported as historical records
R03	`docs/project/DATA_CATALOG.md`	Source lineage, split and count definitions
R04	`runs/afford_pi05_pickup120_20261008_v2_reference/final_motion_audit.json`; `AFFORD_PI05_120_TRAINING.txt`	Final pickup motion audit and result history
R05	`runs/real_only_pi05_20261012_v1_reference/final_closure.json`; `docs/project/REAL_SIM_TRANSFER.md`	Real-only closure and mixed-training provenance
R06	`runs/pi05_real_mixed_comparison_20261012_v1/real_only/metrics.json`; corresponding `mixed/metrics.json`	Matched full-frame offline metrics; accompanying identities/progress/plots
R07	`runs/pi05_real_mixed_closed_loop_20261012_v1_reference/comparison_summary.json`; `delivery_report.json` in the same directory	Per-case outcomes and delivery audit
R08	`runs/real_sim50_pi05_20261015_v1_reference/latest_status_snapshot.json`	Actual 50/50 training and offline evaluation snapshot
R09	`runs/real2sim_held_contact_20261012_v2_reference/summary.json`; `delivery_validation.json` in the same directory; `docs/project/REAL2SIM.md`	Latest held-contact replay artifacts and explanatory history
R10	`runs/isaacsim_three_tray_workflow_20261015_v1_reference/demo12_independent_closure_audit_v1.json`; `demo12_audit_exit.json`	Complete native expert demo and actual audit exit
R11	`runs/isaacsim_fast_stability_20261016_v9_recorded_reference/audit_summary.json`; `case_000/path_stability.json`	Recorded single-pose result and complete-path stability measures
R12	`runs/isaacsim_fixed_pickplace_20261016_v1_reference/efficiency_snapshot_20261016T131354.json`; captured `memory/CURRENT.md`	Dated fixed-pair status; BC restoration and branch navigation
R13	`docs/project/REAL_ROBOT_DEPLOYMENT_20261013.md`; `docs/handoff/examples/`	Deployment history and reviewed server/robot configuration copies
R14	`runs/documentation_handoff_20261016/server_inventory.json`	12:44 SGT read-only environment, configuration and process inventory
R15	`docs/handoff/script_catalog.json`; `docs/project/MODULE_MAP.md`; `docs/handoff/README.md`	Static CLI/source identities and operational navigation
R16	`AGENTS.md`; `docs/governance/MEMORY_POLICY.md`	Collaboration, evidence preservation and archive policy
R17	`runs/documentation_handoff_20261016/validation.json`	Earlier handover link, configuration and CLI checks
R18	`output/pdf/R1Pro_Project_Review_and_Operations_Handover_EN.validation.json`	This edition's PDF, source, inventory and rendering checks

Appendix A. Script inventory

This frozen static inventory contains 232 entry/helper scripts. It covers applicable top-level Python, shell and PowerShell scripts plus package CLI entries. Frozen runs/releases and third-party implementations are not counted again. A listed file is not evidence of successful execution. Historical diagnostics may write data or execute at import time; read source before running them.

Full argument names, default expressions, subcommands, source lines and SHA-256 identities are in `docs/handoff/script_catalog.json` and the frozen companion inventory. AST arguments can belong to separate parsers; do not combine them indiscriminately. The descriptions below are shortened source descriptions.

Native Isaac simulation, data and training (33)

Script / package entry	Source description / inspection clue
`scripts/audit_isaacsim_fast_pickplace.py`	Independent read-only check of physical timing, release and video closure. CLI declarations: 2.
`scripts/audit_isaacsim_three_tray_demo.py`	Independent read-only closure audit of a completed native three-tray demo. CLI declarations: 3.
`scripts/bootstrap_isaac_three_tray_demo.py`	Resume a real controller regression into recorded demo and existing pipeline. CLI declarations: 1.
`scripts/check_isaacsim_clip_seek.py`	Compare accurate input seeks with decoded frame indices; no task labels. CLI declarations: 2.
`scripts/collect_isaac_fixed_pickplace.py`	Separate fixed-base Pick/Place data from physically continuous native pairs. CLI declarations: 1.
`scripts/expand_isaacsim_three_tray.py`	Qualified native variants -> version-2 action3 data -> queued independent BC. CLI declarations: 1.
`scripts/export_isaac_episode_preview.py`	Export only a real fully-qualified source; this is not a training batch. CLI declarations: 3.
`scripts/finalize_isaac_fast_pickplace.py`	Audit closed native cases and concatenate original-speed action previews. CLI declarations: 1.
`scripts/guard_isaacsim_expansion.py`	Detached bounded recovery for the native variant producer, not physics retries. CLI declarations: 1.
`scripts/guard_isaacsim_three_tray_task.py`	Outer guard: bounded coordinator recovery, never retries physical failures. CLI declarations: 2.
`scripts/isaac_mobile.py`	Native mobile19 / PhysX articulation + world D6 drive routing. no static argparse.
`scripts/isaacsim_base_drives.py`	Restore missing D6 drive instances from named source position actuators. no static argparse.
`scripts/isaacsim_control_preflight.py`	Native PhysX control calibration probe, never a task or training source. CLI declarations: 7.

Script / package entry	Source description / inspection clue
`scripts/isaacsim_deformable_coat.py`	Translate the source flex surface to a native PhysX surface-deformable body. no static argparse.
`scripts/isaacsim_kinematic_audit.py`	Compare measured native body-relative FK with the source joint convention. no static argparse.
`scripts/isaacsim_mass_properties.py`	Restore compiled source mass/COM/principal inertia on a NEW native asset. CLI declarations: 3.
`scripts/isaacsim_materials.py`	Restore nominal source sliding friction in a NEW PhysX USD layer. CLI declarations: 3.
`scripts/isaacsim_reference_episode.py`	Re-execute a registered historical expert reference in native PhysX. CLI declarations: 3.
`scripts/isaacsim_scene_smoke.py`	Native Isaac Sim scene import audit; never labels import as task success. CLI declarations: 5.
`scripts/isaacsim_stage_runner.py`	Independent actual wait receipt for one native-workflow child. CLI declarations: 1.
`scripts/prepare_isaacsim_dataset.py`	Validate native completed sources and freeze their three-action index. CLI declarations: 5.
`scripts/prepare_isaacsim_three_tray.py`	Discover the legacy three-tray branch and stage new loaded-handover assets. CLI declarations: 3.
`scripts/prestage_isaacsim_three_tray.py`	Prepare registered held-out assets while a native demo runs independently. CLI declarations: 3.
`scripts/probe_isaacsim_flush_path.py`	Private kinematic regression at an actual failed native state; no physics. CLI declarations: 8.
`scripts/probe_isaacsim_pad_station.py`	Private geometry-only pad-normal station check; never steps physics. CLI declarations: 5.
`scripts/resume_isaacsim_bc_after_video.py`	Resume unchanged frozen Isaac BC only after an explicit native video review. CLI declarations: 1.
`scripts/run_isaacsim_fast_pickplace.py`	Native, original-speed three-second Pick/Place visual benchmark. CLI declarations: 1.
`scripts/run_isaacsim_three_tray_task.py`	Actual native three-tray expert episode; success comes from measured observer. CLI declarations: 6.
`scripts/serve_isaacsim_media.py`	Read-only, single-instance byte-range media service on the existing tailnet. CLI declarations: 4.
`scripts/supervise_isaacsim_fast_pickplace.py`	Detached bounded multi-pose fast video benchmark; never feeds BC implicitly. CLI declarations: 1.
`scripts/supervise_isaacsim_three_tray_assets.py`	Detached, single-instance asset migration stages; not a task/train supervisor. CLI declarations: 6.
`scripts/supervise_isaacsim_three_tray_task.py`	Detached native task -> immutable action3 corpus -> BC supervisor. CLI declarations: 1.

Script / package entry	Source description / inspection clue
scripts/train_pi05_isaac_three_tray.py	BC on frozen, qualified native three-tray sources; no policy execution here. CLI declarations: 3; fixed paths.

Single-tray scenes, control and data (26)

Script / package entry	Source description / inspection clue
scripts/build_single_tray_lab_blender.py	Run with Blender 5.2 --background --python this.py -- --output DIR. CLI declarations: 3.
scripts/collect_three_table_campaign.py	Plan or explicitly collect new complete three-table sources, never train. CLI declarations: 13.
scripts/collect_three_table_full_source.py	Collect a fresh observed three-table task; no training/model launch. CLI declarations: 6.
scripts/export_single_tray_actions.py	Export the new single-tray three-action demonstrations; never starts training. CLI declarations: 4.
scripts/export_three_table_single_tray_actions.py	New mobile/opposed three-table dataset export; no training or relabeling. CLI declarations: 4.
scripts/generate_single_tray_lab.py	Generate a new single-tray photo-reference scene without touching old collections. CLI declarations: 8.
scripts/generate_three_table_single_tray.py	Generate a fresh registered three-table/one-tray variant; no success claim. CLI declarations: 10.
scripts/guard_three_table_workflow.py	Run a bounded outer guardian of an explicitly reviewed workflow process. CLI declarations: 1.
scripts/plan_three_table_reachable_initial_poses.py	Explicit bounded random-initial-pose qualification; never starts physics/training. CLI declarations: 1.
scripts/prepare_single_tray_evaluation_protocol.py	CPU prepare by default. New formal scenes require --generate and full success. CLI declarations: 4.
scripts/probe_single_tray_front_workspace.py	Private static workspace evidence, never a task-success or live-control test. CLI declarations: 6.
scripts/refine_single_tray_lab_robot_visual.py	Blender-only visual face partition; original geometry/collision/inertia untouched. CLI declarations: 2.
scripts/render_single_tray_lab.py	Render a new raw2 replay; failed prefixes remain nontraining evidence. CLI declarations: 4.
scripts/render_three_table_free_run.py	Diagnostic-only native4 continuous replay; never counts as task evaluation. CLI declarations: 3.
scripts/render_three_table_original_trace.py	Native four-camera continuous replay of original raw Trace diagnostics. CLI declarations: 3.

Script / package entry	Source description / inspection clue
scripts/render_three_table_single_tray.py	Render actual mobile three-table source; old one-table traces are rejected. package delegate.
scripts/repair_three_table_source_qualification.py	Additive qualification of closed physically complete sources; no physics rerun. CLI declarations: 3.
scripts/run_single_tray_lab.py	Generate a new, recorded physical single-tray candidate; never train failures. CLI declarations: 29.
scripts/run_single_tray_lab_campaign.py	Durable physical candidate campaign; preserve every failure, never relabel old data. CLI declarations: 3.
scripts/run_three_table_full_candidate.py	Continuous old empty primitives + registered passengers + loaded controller. CLI declarations: 6.
scripts/run_three_table_owned_command.py	Run one explicit command with durable actual PID and exit evidence. CLI declarations: 1.
scripts/run_three_table_single_tray.py	First continuous legacy opposed-pick/place diagnostic; not training data. CLI declarations: 5.
scripts/summarize_single_tray_contact.py	Audit complete-task counts and the new single-tray 3mm/5mm contact gate. CLI declarations: 2.
scripts/supervise_three_action_expansion.py	Independent, bounded source expansion; no training or task-rule changes. CLI declarations: 2.
scripts/supervise_three_table_free_run.py	Independent diagnostic-only model -> timed run -> native media pipeline. CLI declarations: 1.
scripts/supervise_three_table_workflow.py	Detach externally once; this durable workflow waits for actual sources. CLI declarations: 2.

Historical control experiments, diagnostics and audits (67)

Script / package entry	Source description / inspection clue
scripts/analyze_contact_substeps.py	Audit contact chatter by exact private replay of recorded raw/1 controls. CLI declarations: 4.
scripts/analyze_contact_substeps_raw2.py	Audit contact chatter by exact private replay of recorded raw/1 or raw/2 controls. CLI declarations: 4.
scripts/analyze_retained_support_structure.py	Read existing private force CSVs; no simulation or collection writes. no static argparse.
scripts/analyze_support_forces.py	Read-only force reconstruction from complete frozen MuJoCo trace states. CLI declarations: 5.
scripts/audit_case60_sources.py	Read source for interface and side effects. fixed paths.
scripts/audit_case60_third_stage.py	Read source for interface and side effects. fixed paths.

Script / package entry	Source description / inspection clue
scripts/audit_case60_v27_alignment.py	Read source for interface and side effects. fixed paths.
scripts/audit_environment_guards.py	Read source for interface and side effects. fixed paths.
scripts/audit_failures.py	Read-only failure audit. Never changes a collection or labels. CLI declarations: 2.
scripts/audit_fixed_primitives.py	Audit measured primitive timings; does not promote failed episodes. CLI declarations: 2.
scripts/audit_handoff_causes.py	Read-only nested failure audit using small result/event JSON, never HDF5. CLI declarations: 2.
scripts/audit_historical_regression.py	Read-only historical baseline provenance and regression audit. fixed paths.
scripts/audit_load_transfer.py	Read-only command/measurement geometry around an existing push handoff. CLI declarations: 5.
scripts/audit_normalized_scene.py	Read source for interface and side effects. fixed paths.
scripts/audit_regression_first_divergence.py	Read source for interface and side effects. fixed paths.
scripts/audit_yaw_feedback_scope.py	Read-only stage audit of completed production feedback experiments. CLI declarations: 2.
scripts/compare_force_diagnostics.py	Compare all variants over the same elapsed interval, never full-run averages. CLI declarations: 2.
scripts/diagnose_additive_yaw.py	Bounded private push rollout from frozen evidence; never an episode result. CLI declarations: 25.
scripts/diagnose_additive_yaw_certified.py	Bounded private push rollout from frozen evidence; never an episode result. CLI declarations: 25.
scripts/diagnose_alignment_reference.py	Bounded private push rollout from frozen evidence; never an episode result. CLI declarations: 26.
scripts/diagnose_case60_full_open.py	Bounded private push rollout from frozen evidence; never an episode result. CLI declarations: 26.
scripts/diagnose_fresh_case60_shortcut.py	Fresh frozen-v27 case60 episode, selected preflight and one worker only. CLI declarations: 4.
scripts/diagnose_late_suffix_reserve.py	Bounded private push rollout from frozen evidence; never an episode result. CLI declarations: 25.
scripts/diagnose_original_contact_kinematics.py	Bounded private push rollout from frozen evidence; never an episode result. CLI declarations: 25.
scripts/diagnose_production_additive.py	Bounded private push rollout from frozen evidence; never an episode result. CLI declarations: 25.

Script / package entry	Source description / inspection clue
scripts/diagnose_production_withdraw_feedback.py	Bounded private push rollout from frozen evidence; never an episode result. CLI declarations: 25.
scripts/diagnose_push_duration.py	Read-only constructor audit of original zero-motion push path durations. no static argparse.
scripts/diagnose_push_from_trace.py	Bounded private push rollout from frozen evidence; never an episode result. CLI declarations: 25.
scripts/diagnose_reference_rotation.py	Bounded private push rollout from frozen evidence; never an episode result. CLI declarations: 25.
scripts/diagnose_v19_camera_events.py	Read source for interface and side effects. fixed paths.
scripts/diagnose_v19_camera_prediction.py	Read frozen integration states; terminal replay uses recorded per-substep controls. fixed paths.
scripts/diagnose_v19_camera_states.py	Read frozen integration states; terminal replay uses recorded per-substep controls. fixed paths.
scripts/diagnose_v19_camera_yaw.py	Read frozen integration states; terminal replay uses recorded per-substep controls. fixed paths.
scripts/diagnose_v28_buffer_yaw.py	Bounded private push rollout from frozen evidence; never an episode result. CLI declarations: 26.
scripts/diagnose_v28_preview_headroom_fresh.py	Authorized frozen v28 single-field headroom ablation, two fresh scenes. CLI declarations: 3.
scripts/diagnose_withdraw_feedback_runner.py	Bounded private push rollout from frozen evidence; never an episode result. CLI declarations: 25.
scripts/diagnose_withdraw_merge.py	One private withdrawal structure candidate; no production file edits. no static argparse.
scripts/diagnose_withdraw_merge_feedback.py	One private withdrawal structure candidate; no production file edits. no static argparse.
scripts/diagnostic_additive_certificate.py	Strict conservative Bernstein certificates for the SAME additive candidate. no static argparse.
scripts/diagnostic_additive_suffix.py	Private additive yaw correction over an existing validated blended curve. no static argparse.
scripts/diagnostic_contact_kinematics.py	Read-only Jacobian point velocities and contact wrenches; private diagnostics. no static argparse.
scripts/diagnostic_force_suffix.py	Private bounded real-contact force feedback; no production imports this file. no static argparse.
scripts/diagnostic_reference_rotation.py	Read the original planned push curve only; stop before any physical execution. fixed paths.

Script / package entry	Source description / inspection clue
scripts/diagnostic_suffix_reserve.py	One private C2 late-suffix budget allocation; no production import. no static argparse.
scripts/diagnostic_yaw_additive.py	Private one-step yaw amplitude backtracking, retaining every original guard. no static argparse.
scripts/diagnostic_yaw_additive_certified.py	Private one-step yaw amplitude backtracking, retaining every original guard. no static argparse.
scripts/diagnostic_yaw_backtrack.py	Private one-step yaw amplitude backtracking, retaining every original guard. no static argparse.
scripts/diagnostic_yaw_suffix.py	Private one-shot C2 feedback experiment. Never imported by production control. no static argparse.
scripts/inspect_fresh_case60_shortcut.py	Read-only completed fresh case60 versus frozen historical/current baselines. fixed paths.
scripts/inspect_push_verification.py	Read-only release/settling metrics for a frozen final push failure. CLI declarations: 3.
scripts/inspect_v14_grasp_failures.py	Small read-only snapshots of the alternate-rim ablation failures. CLI declarations: 2.
scripts/inspect_v21_case6_direct_geometry.py	Read source for interface and side effects. fixed paths.
scripts/inspect_v21_case6_release.py	Read source for interface and side effects. fixed paths.
scripts/inspect_v21_other_rims.py	Read source for interface and side effects. fixed paths.
scripts/inspect_withdraw_geometry.py	Read-only final reserved footprint and gap audit. no static argparse.
scripts/probe_airport_motion.py	Isolated diagnostic attempt: never publishes a training dataset. CLI declarations: 4.
scripts/probe_alternate_rims.py	Read-only failed-state qualification of both physical rim pairs. CLI declarations: 4.
scripts/probe_constant_push_height.py	Private kinematic-only command-continuous retained-hand push candidate. CLI declarations: 2.
scripts/probe_fixed_primitives.py	Snapshot-initialized physics FIXTURE, never a training episode or full-task success. CLI declarations: 7.
scripts/summarize_command_ablation.py	Compact read-only completed-result comparison; never infer partial success. CLI declarations: 4.
scripts/summarize_yaw_diagnostic.py	Read frozen yaw trace and already completed private candidate geometry. fixed paths.
scripts/test_diagnostic_additive_certificate.py	Read source for interface and side effects. no static argparse.
scripts/test_diagnostic_additive_suffix.py	Read source for interface and side effects. no static argparse.

Script / package entry	Source description / inspection clue
scripts/test_diagnostic_contact_kinematics.py	Read source for interface and side effects. no static argparse.
scripts/test_diagnostic_force_suffix.py	Read source for interface and side effects. no static argparse.
scripts/test_diagnostic_yaw_backtrack.py	Bounded retry semantics for the private candidate, without physical tuning. no static argparse.
scripts/test_diagnostic_yaw_suffix.py	Standalone mathematical checks; no MuJoCo controller changes. no static argparse.

Monitoring and indexes - verify actual side effects (9)

Script / package entry	Source description / inspection clue
scripts/collect_vla_training_status.py	Read-only metadata snapshot; never starts training or downloads video. CLI declarations: 4.
scripts/maintain_three_table_server_index.py	Read-only run metadata index; never controls jobs or certifies media. CLI declarations: 4.
scripts/register_video_index_task.ps1	Read source for interface and side effects. no static argparse.
scripts/watch_isaacsim_episode_preview.py	Independently expose action clips as soon as a full native source closes. CLI declarations: 1.
scripts/watch_pi05_mix50.py	Read-only monitor of the frozen 50/50 run; no control or training commands. CLI declarations: 2.
scripts/watch_pi05_real_only.py	Read-only progress monitor for the isolated real-only experiment. CLI declarations: 2.
scripts/watch_real_sim_transfer.py	Read-only real/sim transfer stage, percent, actual MSE and measured ETA. CLI declarations: 2.
scripts/watch_three_table_data_version2.py	Start independent dataset v2 export during actual milestone10 model evaluation. CLI declarations: 3.
scripts/watch_vla_comparison.py	Read-only comparison progress; no model imports and no fabricated ETA. CLI declarations: 3.

Installation, maintenance and legacy multi-tray baseline (42)

Script / package entry	Source description / inspection clue
r1pro_teleop/airport_agent.py	Callable simulation collection agent. Does not impersonate a human or navigate a fixed base. CLI declarations: 16; subcommands.
r1pro_teleop/airport_arrange.py	Closed-loop program-generated tray arrangement in simulation only. CLI declarations: 3.
r1pro_teleop/airport_fsm/__main__.py	python -m r1pro_teleop.airport_fsm --help CLI declarations: 25; subcommands.

Script / package entry	Source description / inspection clue
r1pro_teleop/airport_scenes.py	Versioned, deterministic airport tray scene generation; no controller changes. CLI declarations: 6.
r1pro_teleop/airport_service.py	Airport Quest service: authenticated existing protocol, explicit episode lifecycle. CLI declarations: 5.
r1pro_teleop/build_assets.py	Build compound convex collisions and a dynamic MJCF, preserving source assets. CLI declarations: 2.
r1pro_teleop/dataset.py	Validate raw synchronized episodes; export through the official LeRobot API. CLI declarations: 5.
r1pro_teleop/experiments/__main__.py	python -m r1pro_teleop.experiments --help CLI declarations: 30; subcommands.
r1pro_teleop/quality.py	Offline data review and conservative tray-task checks; never relabel demonstrations. CLI declarations: 2.
r1pro_teleop/quest_bridge.py	Forward fresh Oculus Reader APK packets via authenticated HTTP to simulation. CLI declarations: 5.
r1pro_teleop/quest_relay.py	Loopback-only authenticated USB relay for Quest Browser WebXR + video. CLI declarations: 6.
r1pro_teleop/replay.py	Replay saved actions through physics, compare recorded states, and export a review video. CLI declarations: 4.
r1pro_teleop/server.py	HTTP viewer/control service for MuJoCo only. No ROS or hardware outputs. CLI declarations: 7.
r1pro_teleop/smoke.py	No-headset acceptance checks: physics, contact cavity, rendering and recording. CLI declarations: 3.
r1pro_teleop/vla/data_benchmark.py	Small isolated stage timings for native-to-training video conversion. CLI declarations: 5.
r1pro_teleop/vla/data_lineage.py	Read-only source and frame verification for additive BC warm starts. CLI declarations: 3.
r1pro_teleop/vla/data_native_pipeline.py	Supervise a new immutable native-video training conversion on the server. CLI declarations: 6.
r1pro_teleop/vla/data_quality_audit.py	Read-only skill/qualification coverage audit for a prepared training manifest. CLI declarations: 3.
r1pro_teleop/vla/data_skill_coverage.py	Full-frame source audit and immutable constituent coverage for skill merges. CLI declarations: 3.
r1pro_teleop/workspace_check.py	Headless IK reachability probe; no contacts, task execution, or real robot control. CLI declarations: 2.
scripts/build_handoff_catalog.py	Build a static handoff inventory without importing or executing project jobs. CLI declarations: 2; fixed paths.

Script / package entry	Source description / inspection clue
`scripts/check_r1pro_runtime.py`	Record actual Linux GPU/runtime identities and test CUDA plus EGL rendering. no static argparse.
`scripts/collect_airport_batches.py`	Bounded, resumable generation -> native export -> report, using frozen code. CLI declarations: 5.
`scripts/collect_failure_videos.py`	Bounded, resumable native failure evidence rendering and SSH collection. CLI declarations: 18; subcommands; fixed paths.
`scripts/control_ablation.py`	Same-code, same-seed fresh physical ablation; no cross-run state reuse. CLI declarations: 6.
`scripts/control_regression.py`	Fresh server control evaluation on the unchanged 400-scene source and split. CLI declarations: 5.
`scripts/deploy_experiments.py`	Server-side fresh release deployment; invoke with a non-base Conda Python. CLI declarations: 3.
`scripts/experiment_server_smoke.py`	Bounded server smoke: core tests precede this script; always writes status/evidence. CLI declarations: 3.
`scripts/export_project_interactions.py`	Export project conversation messages from local logs, never model reasoning/tools. CLI declarations: 3.
`scripts/generate_airport_scenes.py`	Read source for interface and side effects. package delegate.
`scripts/native32_expansion_pipeline.py`	Two independent native replay workers, immutable source/reuse, no training. fixed paths.
`scripts/preview_airport_success.py`	Static inspection images of saved physics states; not a training replay. CLI declarations: 4.
`scripts/project.py`	Portable project checks and immutable four-camera scene preparation. CLI declarations: 4; subcommands.
`scripts/repair_airport_initial_settle.py`	Fork scenes; repair floating stacked initializations before collection only. CLI declarations: 2.
`scripts/run_r1pro_gpu.sh`	Read source for interface and side effects. no static argparse.
`scripts/run_real_sim_pickplace.py`	Offline real/sim transfer experiment; no real robot command interface. CLI declarations: 3; fixed paths.
`scripts/select_r1pro_gpu.py`	Map nvidia-smi's physical GPU index to EGL's independent device enumeration. CLI declarations: 2.
`scripts/setup_r1pro_server.sh`	Read source for interface and side effects. no static argparse.
`scripts/smoke_fourcam_lerobot.py`	Stationary camera/loader integration fixture, never a task demonstration. CLI declarations: 3.

Script / package entry	Source description / inspection clue
scripts/sync_task_spec.py	Refresh/check the implementation appendix inside the single task specification. CLI declarations: 2.
scripts/upgrade_airport_fourcam.py	Fork a scene manifest with four measured-intrinsic pinhole cameras. CLI declarations: 3.
scripts/validate_fourcam_scene.py	Render four native views and verify camera-only upgrade preserves physics. CLI declarations: 3.

Data, VLA training and policy evaluation (43)

Script / package entry	Source description / inspection clue
scripts/close_pi05_comparison_media.py	Read-only validation of paired model videos/traces, then delivery metadata. CLI declarations: 1.
scripts/compare_pi05_closed_loop.py	Paired held-out simulation Pick/Place; direct model controls, no fallback. CLI declarations: 2; fixed paths.
scripts/compare_pi05_real_mixed_plots.py	Paired all-frame real-test inference and scientific comparison plots. CLI declarations: 2; fixed paths.
scripts/compare_pickup120.py	Isolated server BC comparison: exactly120 pickup episodes, same50 heldout poses. CLI declarations: 2; fixed paths.
scripts/convert_vla_lerobot.py	Convert verified train spans to a new LeRobot v2 pi0.5 training dataset. CLI declarations: 5.
scripts/diagnose_pi05_skills.py	Isolated skill diagnostics from verified expert physics initial snapshots. CLI declarations: 19.
scripts/evaluate_action8_checkpoint.py	Durable single-milestone evaluation; trainer owns the outer resume chain. CLI declarations: 5.
scripts/evaluate_action8_checkpoint_dual.py	Optional static dual-GPU full-test evaluation. Serial supervisor is unchanged. CLI declarations: 5.
scripts/evaluate_pi05_airport.py	Run fresh pi05 MuJoCo evaluations or train-only private expert DAgger queries. CLI declarations: 31.
scripts/evaluate_single_tray_pi05.py	Evaluate one existing single-tray scene with a real checkpoint service. CLI declarations: 9.
scripts/evaluate_single_tray_pi05_batch.py	Explicit new-task milestone protocol; GPU execution requires --execute. CLI declarations: 10.
scripts/evaluate_three_table_single_tray_pi05.py	Pure model mobile19 evaluation of one new three-table scene. CLI declarations: 8.
scripts/evaluate_three_table_single_tray_pi05_batch.py	Explicit new-task milestone protocol; GPU execution requires --execute. CLI declarations: 10.
scripts/generate_pickup_lerobot.py	Independent parallel pickup-only teacher data, single-writer LeRobot v2.1. CLI declarations: 8; fixed paths.

Script / package entry	Source description / inspection clue
scripts/materialize_complete_task_actions.py	Export complete action windows without starting training or a GPU model. CLI declarations: 9.
scripts/materialize_complete_task_actions_parallel.py	Bounded isolated CPU case exporters; reuse only independently verified cases. CLI declarations: 11.
scripts/merge_vla_skill_dataset.py	Create a new skill-contiguous derivative without rewriting source data. CLI declarations: 5.
scripts/observe_task_complete_sources_parallel.py	Versioned router handoff around an existing owned native producer; no renderer signals. CLI declarations: 4; fixed paths.
scripts/plan_complete_task_actions.py	Read-only CPU planning of whole actions from complete simulation tasks. CLI declarations: 4.
scripts/plot_pi05_existing_summary.py	Plot verified existing aggregate reports while paired inference is pending. no static argparse.
scripts/prepare_action8_evaluation_protocol.py	CPU-only immutable preregistration of all original held-out test scenes. CLI declarations: 3.
scripts/prepare_vla_dataset.py	Build frame-aligned pi0.5 supervision manifests without modifying collections. CLI declarations: 5.
scripts/prepare_vla_skill_dataset.py	Exclusive new skill-derived dataset, quality/coverage audit and train preflight. CLI declarations: 8.
scripts/render_action8_evaluation.py	Four native cameras beside a fresh VLA attempt's trace; inspection only. CLI declarations: 4.
scripts/render_task_complete_sources.py	Render immutable complete source tasks; benchmark first, explicit batch dispatch later. CLI declarations: 11.
scripts/render_vla_native_sources.py	Replay frozen successful episodes into new, verified native four-camera video files. CLI declarations: 3; fixed paths.
scripts/render_vla_rollout_videos.py	Native diagnostic heads from actual learned-controller traces; no training export. CLI declarations: 6.
scripts/rerender_pickup_camera_shells.py	New video/data version of immutable pickup traces; renderer-only shell hide. CLI declarations: 5.
scripts/resume_afford_pi05_comparison.py	Continue the failed comparison in a new run, preserving the old evidence. CLI declarations: 2; fixed paths.
scripts/run_pi05_real_only.py	Real-only BC and offline held-out action evaluation; never controls a robot. CLI declarations: 2; fixed paths.
scripts/run_three_table_pi05_free_run.py	Independent free-run diagnostic, never a formal evaluator or dataset source. CLI declarations: 7.

Script / package entry	Source description / inspection clue
scripts/run_vla_fixed_scene.py	One prompt, one fixed scene, timed pure-VLA simulation and native videos. CLI declarations: 10.
scripts/serve_pi05_airport.py	Serve an actually saved airport pi0.5 fine-tune using official openpi. CLI declarations: 5.
scripts/serve_pi05_diagnostic.py	Explicit nonproduction pi05 checkpoint serving for diagnostic physics smoke. CLI declarations: 4.
scripts/supervise_action8_training.py	Launch a durable complete-source-gated training/evaluation chain. CLI declarations: 3.
scripts/supervise_single_tray_training.py	Launch the independent new single-tray three-action data/BC pipeline. CLI declarations: 2.
scripts/supervise_three_table_single_tray_training.py	Explicit fresh three-table/mobile/opposed task pipeline. No old-task reuse. CLI declarations: 3.
scripts/supervise_verified_three_action_training.py	Independent guardian for a frozen, completed three-action BC dataset. CLI declarations: 3.
scripts/train_pi05_accelerated.py	Project cache and bounded milestone training; no automatic evaluation service. CLI declarations: 6.
scripts/train_pi05_airport.py	Run project-owned pi05 mobile19 LoRA extension inside a non-base Conda env. CLI declarations: 5.
scripts/train_pi05_real_sim50.py	Independent 50/50 BC release; reuses immutable validated real/sim caches. CLI declarations: 2; fixed paths.
scripts/transition_task_complete_sources_parallel.py	Reviewed live-publication transition; immutable source/video/per-case evidence retained. CLI declarations: 2; fixed paths.
scripts/vla_fixed_scene_commands.sh	Run on the server after copying the standalone diagnostic files into project. fixed paths.

Offline real-command replay and registration (7)

Script / package entry	Source description / inspection clue
scripts/audit_real2sim.py	Audit every original real episode against an explicit simulation model. CLI declarations: 3.
scripts/build_real2sim_finger_collision.py	Decompose project-owned URDF meshes into portable convex collision parts. CLI declarations: 1.
scripts/diagnose_real2sim_held_pose.py	Bounded, offline Place registration diagnosis; never changes source commands. CLI declarations: 2.
scripts/fit_real2sim_registration.py	Offline initial held-tray pose fit to native collision geometry, no dynamics. CLI declarations: 4.

Script / package entry	Source description / inspection clue
`scripts/fit_real2sim_torso.py`	Finite offline free-space torso servo fit; never counted as task success. CLI declarations: 1.
`scripts/replay_real2sim.py`	Replay original real Pick/Place in a separate full-joint model; simulation only. CLI declarations: 11.
`scripts/validate_real2sim.py`	Independently verify a completed replay's clocks, pose, trace and videos. CLI declarations: 2.

Robot deployment and transport (5)

Script / package entry	Source description / inspection clue
`scripts/control_galaxea_servers.py`	Workstation keyboard command-stop; NOT a hardware emergency stop. CLI declarations: 2.
`scripts/forward_galaxea_lan.py`	Private wired TCP relay; preserves existing model and robot safety protocol. no static argparse.
`scripts/galaxea_client_oneclick.sh`	New convenience entry; no changes to robot run/scheduler/filter. no static argparse.
`scripts/run_galaxea_client_record_compat.py`	New robot entry; preserves original run/scheduler/filter/recorder files. CLI declarations: 1.
`scripts/serve_pi05_galaxea_robot.py`	New workstation endpoint for unchanged Galaxea robot run.py. CLI declarations: 1; fixed paths.

Appendix B. Configuration inventory

The frozen inventory contains 132 top-level configuration files. Their names identify experiment versions; presence does not imply that a recipe matches the current task. Preserve the chosen file's SHA and inspect all fixed paths before reuse. The separate reviewed deployment examples are listed after the inventory.

Configuration path	SHA-256 prefix
configs/airport_ablation_alternate.json	39764323b8716d39
configs/airport_ablation_baseline.json	e621cdaa98e2273a
configs/airport_ablation_payload.json	404f16d08fc62c97
configs/airport_ablation_substep.json	cf0112a3fe5693b6
configs/airport_approximate_completion_conservative_ik_retry3.json	566c62e50addae89
configs/airport_approximate_completion_legacy_handoff_retry3.json	73cc523834d2d33d
configs/airport_approximate_completion_retry3.json	1c384a119f42484e
configs/airport_approximate_completion_supported_push_retry3.json	945e5f20849e3d42
configs/airport_arrival_alternate_align.json	a42c82ca4c492cf9
configs/airport_arrival_alternate_arrival.json	5bedc904f4865cd7
configs/airport_arrival_alternate_feedback.json	91e200a9141ce884
configs/airport_arrival_arrival.json	4d51985fe30c9b6f
configs/airport_arrival_baseline.json	6c3940d8b52ef2a6
configs/airport_buffer_baseline.json	0bd9b7e4b8bc8023
configs/airport_buffer_command_feedback.json	57a7cdfb5901af1f
configs/airport_buffer_geometry.json	7a177abd317b1c23
configs/airport_buffer_hybrid.json	2da5ce9f2cf445c2
configs/airport_buffer_paths.json	1990ede16258a10b
configs/airport_certified_additive.json	593a86dcc1d7e6c2
configs/airport_certified_baseline.json	1f336d1bcff79673
configs/airport_certified_combined.json	b9deb3dd6a0528a3
configs/airport_certified_geometry.json	03cff2db7affbe7f
configs/airport_certified_guarded_reference.json	d14f7e6de9c7ab96
configs/airport_clock_float_baseline.json	2fc293b502d283ed
configs/airport_clock_step_baseline.json	201277b76981559b
configs/airport_clock_step_preview.json	d9101768b0380c44
configs/airport_clock_step_shortcut.json	f805e6c176db6e1c

Configuration path	SHA-256 prefix
configs/airport_completion_direct_ik80_retry3.json	fc04f689aac4c7ed
configs/airport_completion_direct_lift_nominal_retry3.json	f7260d691d71e36a
configs/airport_completion_direct_retry3.json	71e8063855dde50a
configs/airport_completion_first.json	c5ebdf54636369ee
configs/airport_completion_flow_retry3.json	cb4d83d24c5643b7
configs/airport_completion_free_open_retry3.json	155cf0c28c4d4209
configs/airport_completion_handoff_clearance_retry3.json	47401cc363e775e6
configs/airport_completion_ik.json	01e5e56692751aeb
configs/airport_completion_ik40.json	f59afc19ed8cc03d
configs/airport_completion_ik40_retry3.json	5952169db3a8def4
configs/airport_completion_ik80_retry3.json	04f61d0182837918
configs/airport_completion_lift_nominal_retry3.json	056547f95d31c6da
configs/airport_completion_lift_retry3.json	81a69d949fe1c147
configs/airport_completion_nominal_retry3.json	1550531c4a173329
configs/airport_completion_orientation_retry3.json	38998238e1195c4a
configs/airport_completion_priority_retry3.json	cb943403e5e0273c
configs/airport_completion_priority_speed_retry3.json	c419ebadd93be797
configs/airport_completion_release_orientation_retry3.json	2432c57dc7a1ff24
configs/airport_completion_release_retry3.json	295cb5e37b78b281
configs/airport_completion_reservation_retry3.json	e8e4a956800174bb
configs/airport_completion_retry3.json	6205f51006a717d7
configs/airport_completion_stage_extended_base_retry3.json	192073f3d5b46d0a
configs/airport_completion_stage_joint_endpoint_pos100_retry3.json	792023cca6e12a12
configs/airport_completion_stage_joint_endpoint_retry3.json	3fa3283afcf4d416
configs/airport_completion_stage_multistart_joint_endpoint_retry3.json	42ad2f474cf38d9e
configs/airport_completion_stage_multistart_retry3.json	1d7f0f9a7c7e9c4e
configs/airport_completion_stage_priority_retry3.json	5ee37a739059f681
configs/airport_completion_stage_priority_speed_retry3.json	52cd326fcb4e27e4
configs/airport_completion_stage_private_pos090_retry3.json	d9155ed1d37dba07
configs/airport_completion_stage_private_pos100_extended_base_retry3.json	9d2353a585ccae9c
configs/airport_completion_stage_private_pos100_retry3.json	af8b64d58eeffbc6

Configuration path	SHA-256 prefix
configs/airport_completion_stage_refit_free_first_retry3.json	d57e36d0bd22f9f6
configs/airport_completion_stage_refit_retry3.json	d43b7d036576e8e1
configs/airport_completion_stage_refit_subdivision_retry3.json	4e5a0a017e2417da
configs/airport_completion_stage_subdivision_retry3.json	5d649479299af36b
configs/airport_completion_stage_supported_combined_screen_retry3.json	574686d157fdcba6
configs/airport_completion_stage_supported_staging_retry3.json	180e0e71aec94ebb
configs/airport_completion_task_retry3.json	6e975e838a7cb39d
configs/airport_destination_combined.json	83dea38c15a00b6d
configs/airport_destination_push_command.json	7880050be4a17690
configs/airport_destination_push_zero.json	495629387bf5a5ff
configs/airport_destination_screen_zero.json	a374c4997e354c24
configs/airport_feedback_baseline.json	bb4ba563680ff26a
configs/airport_feedback_combined.json	616647a5b7d22443
configs/airport_feedback_command_feedback.json	5322c68a25fe05cc
configs/airport_feedback_feedback.json	57ab81e43394c4dd
configs/airport_feedback_geometry.json	ed3cb9149358632c
configs/airport_geometry_baseline.json	25c05d023987c7ef
configs/airport_geometry_combined.json	26b175c17ea067e5
configs/airport_geometry_destination.json	4250b6955746e342
configs/airport_geometry_pickup.json	1793ed9860b62836
configs/airport_merge_additive.json	d8267fba6155c069
configs/airport_merge_baseline.json	b25746cda102a267
configs/airport_merge_combined.json	38fb044650aa878f
configs/airport_merge_combined_merge.json	5262d765fff772e8
configs/airport_merge_preview_merge.json	41cc6b55c2a9f1cf
configs/airport_preview_baseline.json	6af1b072f6fd91f2
configs/airport_preview_combined.json	5b2e4ca6971ba162
configs/airport_preview_geometry.json	b8f79325230721ad
configs/airport_preview_guarded.json	2862a22e0a218aa6
configs/airport_preview_paths.json	9b36a3f06a701106
configs/airport_primitives3.json	14abc0388f0dc4f2

Configuration path	SHA-256 prefix
configs/airport_primitives3_smoke.json	7140ea2a22dc0682
configs/airport_reach_pull.json	d671ec154b2220e3
configs/airport_reference_baseline.json	6faafd6916f3bdd7
configs/airport_reference_command.json	1ebd98ed89b39ce0
configs/airport_reference_command_substep.json	405394af3dd6f03f
configs/airport_reference_geometry.json	ef66b7ddfba58836
configs/airport_reference_geometry_preserve.json	e439e7ba2560ecb5
configs/airport_reference_paths_feedback.json	cb0610fdf534538a
configs/airport_reference_preserve.json	4579a51d3330b0c0
configs/airport_retiming_additive.json	504138f4019b74f0
configs/airport_retiming_baseline.json	50c8c09ab8ec1ffd
configs/airport_retiming_combined.json	68f4f4f596dd1f73
configs/airport_retiming_preview.json	2696d79c95afe64f
configs/airport_retiming_shortcut.json	4432dfef0466e8eb
configs/airport_robust_v10.json	3b9ec3b74542d493
configs/airport_robust_v11.json	829ff55ee5cadc66
configs/airport_robust_v12.json	491c1d435f49375b
configs/airport_robust_v13.json	76ba877796db7432
configs/airport_scoped_baseline.json	3ef1e38821a91e36
configs/airport_scoped_feedback_all.json	53fde33f9dc5f1eb
configs/airport_scoped_feedback_placement.json	434312f6e5c209fb
configs/airport_scoped_paths.json	60af3e883190d449
configs/airport_target_baseline.json	e212bc64f1489862
configs/airport_target_combined.json	ebf47adb52d0a50b
configs/airport_target_geometry.json	cf5ea883f1202de6
configs/airport_target_geometry_feedback.json	81d29bd7ac8337e0
configs/airport_target_geometry_reference.json	b7a374ac56cf4e50
configs/experiment_candidates.json	55a50b2e5d4e4c98
configs/experiment_search_space.json	6f34cafec6540fde
configs/r1pro_verified_limits_TEMPLATE.json	b16f846d34e66802
configs/vla_pi05_action8_accelerated_v1.json	a997b3915d26e31f

Configuration path	SHA-256 prefix
configs/vla_pi05_airport_bootstrap4_lora.json	a7e1209b51e259df
configs/vla_pi05_airport_bootstrap4_lora_v2.json	31786527493b94a8
configs/vla_pi05_airport_native16_v3_lora.json	a35dd4b081349611
configs/vla_pi05_airport_native32_skill_only6_dualgpu_20261005.json	ffad8c5886a818d6
configs/vla_pi05_airport_native32_skillmerged_dualgpu_dp_20261005.json	fb4ae809bbe1afb1
configs/vla_pi05_airport_native32_skillmerged_dualgpu_warmstart_20261005.json	fb4ae809bbe1afb1
configs/vla_pi05_airport_native32_v2_warmstart_lora.json	1b212bfea7f6be1f
configs/vla_pi05_airport_native32_v3_warmstart_lora.json	8512aca5fcb3d68b
configs/vla_pi05_airport_native32_v4_skillmerged_warmstart_lora.json	7d50824af7977de2
configs/vla_pi05_single_tray_action3_v1.json	c79845b20829de1e
configs/vla_pi05_three_table_single_tray_action3_v1.json	5382711454c74f09
configs/vr_teleoperation_whole_body_launch.py	2da5aeb841606850

Reviewed deployment and stage examples

docs/handoff/examples/mixed.robot.lan.example.toml

docs/handoff/examples/mixed.server.example.json

docs/handoff/examples/real_only.robot.lan.example.toml

docs/handoff/examples/real_only.server.example.json

docs/handoff/examples/stage_job.example.json

The examples contain configuration structure and model identities. Replace new-output placeholders and verify the actual workstation/robot configuration before use. They do not contain credentials.